



UNIVERSIDAD ESTATAL A DISTANCIA VICERRECTORÍA ACADÉMICA

ESCUELA DE CIENCIAS EXACTAS Y NATURALES

CARRERA INGENIERÍA INFORMÁTICA

CÁTEDRA INGENIERÍA DE SOFTWARE ASIGNATURA

03301 ANÁLISIS Y DISEÑO DE SISTEMAS

PROYECTO 3

VALOR: 3.0 %

ESTUDIANTE:

FRANCISO CAMPOS SANDI

IDENTIFICACIÓN: 114750560

TUTOR:

ENDRIK MEJÍAS MOREIRA

GRUPO 04

CENTRO UNIVERSITARIO:

San Vito

II CUATRIMESTRE, 2025



Contenido

Introducción.....	3
Desarrollo.....	5
Respuesta 1	5
¿Cuál se aplica mejor al caso?	9
Respuesta pregunta 2	11
Respuesta 3	12
Respuesta 4	14
Respuesta 5	15
Respuesta 6	17
Principio de Responsabilidad Única (Single Responsibility Principle)	17
Principio de Abierto/Cerrado (Open/Closed Principle)	18
Principio de Sustitución de Liskov (Liskov Substitution Principle)	18
Principio de Inversión de Dependencia.....	18
Principio de inversión de dependencia (D - Dependency Inversion Principle) ...	19
Respuesta 7	19
Conclusión	21
Bibliografía	23

Índice de ilustraciones

Ilustración 1: Arquitectura Cliente-Servidor.....	8
Ilustración 2: Arquitectura en capas.....	8
Ilustración 3: Modelo-Vista-Controlador.....	9
Ilustración 4: Story Mapping	15

Índice de tablas

Tabla 1: Patrones Arquitectónicos	5
Tabla 2: Manejo de riesgos.....	15

Introducción

El presente trabajo aborda el análisis integral del diseño de sistemas aplicados al proyecto ficticio “Red de Bibliotecas Interconectadas” (RedBiln), desarrollado en el contexto de la asignatura de Ingeniería de Software, el objetivo principal es estudiar las soluciones arquitectónicas, técnicas y metodológicas que podrían implementarse para satisfacer las necesidades funcionales y operativas de una red nacional de bibliotecas

La relevancia del tema radica en el creciente interés por digitalizar procesos educativos y culturales, especialmente aquellos relacionados con el acceso universal a la lectura, además, la estructuración lógica de sistemas interconectados como RedBiln permite explorar diversos enfoques en la ingeniería de software, desde patrones arquitectónicos hasta modelado de historias de usuario, evaluación de riesgos y aplicación de estándares de calidad.

El contenido del trabajo se compone de un análisis comparativo de patrones arquitectónicos, diseño de diagrama UML, modelado de historias de usuario, identificación de riesgos y aplicación del estándar ISO/IEC 25010:2011, además, se han incorporado citas verificables que fortalecen la validez conceptual de cada argumento.

La metodología empleada se basó en el estudio crítico de los requisitos planteados en las indicaciones del proyecto, combinando interpretación técnica con propuestas concretas de solución, el enfoque reflexivo permitió evidenciar cómo se relacionan las decisiones de diseño con los principios éticos, la funcionalidad esperada y la experiencia del usuario, en conjunto, el trabajo articula teoría, práctica y contexto, ofreciendo una visión completa sobre el desarrollo de sistemas informáticos con impacto social.

Desarrollo

Respuesta 1

Tabla 1: *Patrones Arquitectónicos*

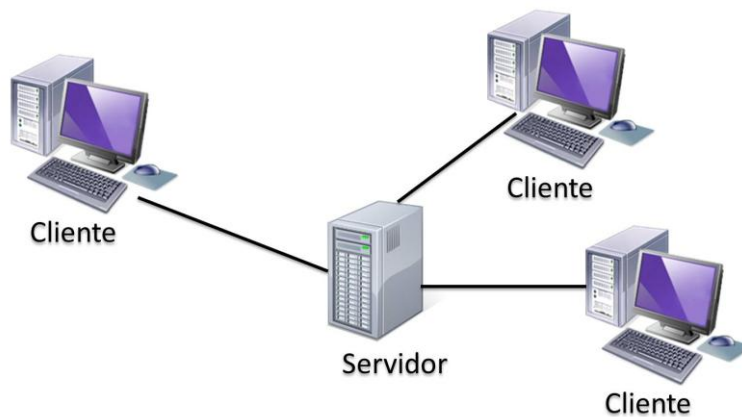
Patrón Arquitectónico	Descripción	Ventajas	Desventajas
Modelo-Vista-Controlador	Este patrón busca separar responsabilidades en tres componentes principales, promoviendo una estructura clara para el manejo de la interfaz, la lógica y los datos, "es un patrón de diseño arquitectónico de software, que sirve para clasificar la información, la lógica del sistema y la interfaz que se le presenta al usuario"(García, 2017, párr.01) esta separación permite que cada componente evolucione de forma independiente, facilitando la reutilización y el mantenimiento del sistema.	• Favorece la escalabilidad y mantenibilidad del sistema. • Facilita el desarrollo colaborativo gracias a la separación de responsabilidades.	• Puede implicar una curva de aprendizaje mayor para desarrolladores nuevos. • Requiere una adecuada sincronización entre las partes para evitar inconsistencias.

Arquitectura en Capas	Esta arquitectura organiza el sistema en niveles jerárquicos, donde cada capa tiene una responsabilidad específica y se comunica con la capa inmediatamente inferior o superior, "es una de las técnicas más comunes que los Diseñadores/ Desarrolladores de software utilizan para separar un sistema de software muy complejo, principalmente para aplicaciones empresariales."(RJ Code Advance, 2022, párr.01), gracias a esta estructura, los sistemas complejos se vuelven más comprensibles, reutilizables y modularizables, lo cual es ideal para ambientes empresariales.	• Proporciona una separación clara entre lógica de presentación, lógica de negocio y datos. • Permite escalar y modificar capas sin alterar otras.	• Puede generar sobrecarga en el procesamiento por el número de capas. • La dependencia entre capas puede dificultar la flexibilidad si no se gestiona adecuadamente.
Arquitectura Cliente-Servidor	Esta arquitectura define roles específicos entre dos entidades: el cliente, que solicita servicios, y el servidor, que los provee. Se	• Facilita la centralización de datos y servicios. • Permite el	• Dependencia constante de la conexión entre cliente y servidor. • Vulnerabilidad ante

	ha convertido en un estándar para aplicaciones distribuidas, "se llama así a toda arquitectura en la que participan dos componentes: uno es el cliente que utiliza unos servicios, y otro es el servidor que proporciona esos servicios"(García de Zúñiga, 2025, párr.02), su simplicidad de roles permite una clara estructuración del flujo de información y facilita la implementación de servicios en red como el acceso a bibliotecas interconectadas.	acceso remoto desde múltiples dispositivos.	caídas del servidor o sobrecarga en su procesamiento.
--	--	--	--

Nota: Fuente Elaboración propia (2025).

Ilustración 1: Arquitectura Cliente-Servidor

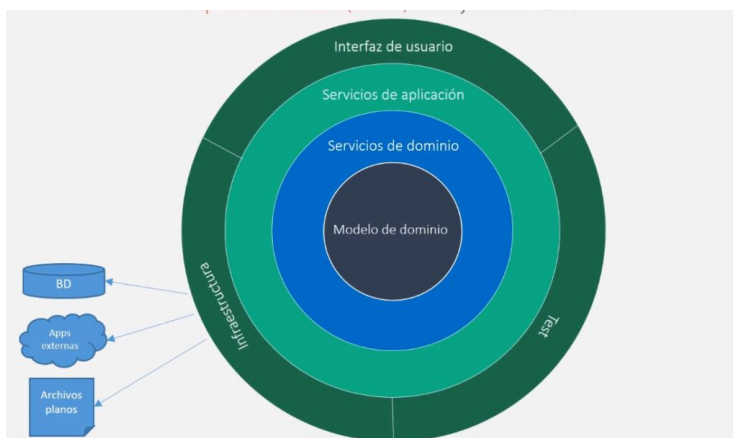


La información se comparte fácilmente a través de redes, en donde los servidores almacenan datos que pueden ser compartidos por los clientes.

Fuente: Tomado de *Arquitectura Cliente Servidor Ejemplos* [Imagen]. (2022).

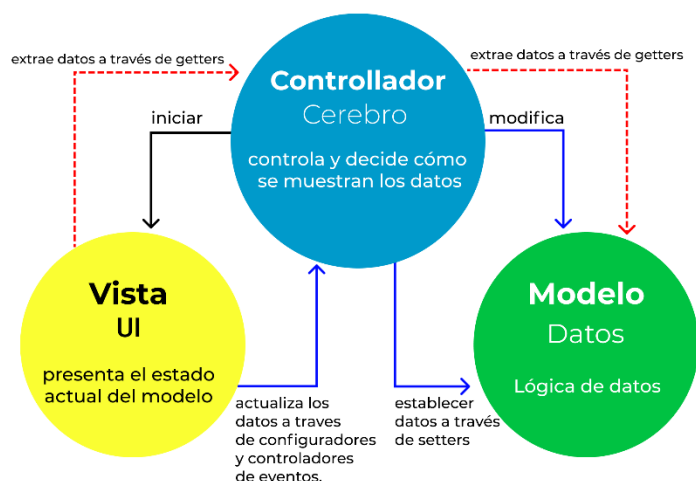
Arbol. <https://i.ytimg.com/vi/xjJVqfzR-fg/maxresdefault.jpg>

Ilustración 2: Arquitectura en capas



Fuente: Tomado de RJ Code Advance. (2020). *Arquitectura en Capas* [Imagen]. <https://i2.wp.com/rjcodeadvance.com/wp-content/uploads/2019/10/image-7.png?fit=1024,602&ssl=1>

Ilustración 3: Modelo-Vista-Controlador



Fuente: Tomado de D. Hernandez, R. (2023). *El patrón modelo-vista-controlador: Arquitectura y*

frameworks explicados [Imagen]. <https://www.freecodecamp.org/espanol/news/content/images/size/w1600/2021/06/MVC3.png>

¿Cuál se aplica mejor al caso?

Para el desarrollo de la solución propuesta para la Red de Bibliotecas Interconectadas (RedBiln), el diseño arquitectónico más apropiado es una arquitectura en capas complementada con principios del modelo cliente-servidor. Esta decisión no se toma de manera arbitraria, sino que se fundamenta en los requerimientos funcionales, el alcance geográfico y la necesidad de escalabilidad del sistema.

Por un lado, la arquitectura en capas permitiría estructurar el software en distintos niveles: presentación, lógica de negocio, acceso a datos y almacenamiento, esto facilitaría la separación de responsabilidades y el mantenimiento del sistema, por ejemplo, toda la interfaz de usuario, tanto de la aplicación móvil como del portal web, se gestionaría desde la capa de presentación, mientras que la lógica para gestionar préstamos, reservas o

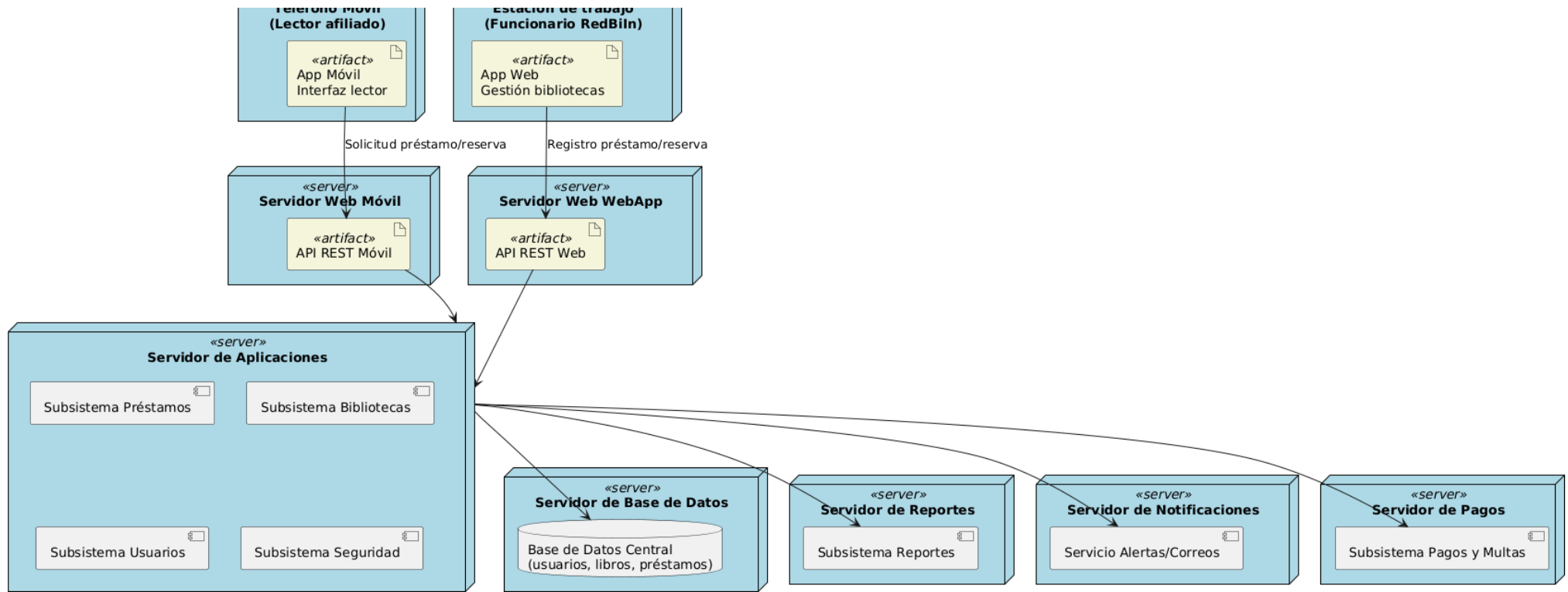
traslados quedaría en la capa de negocio, esto mejora la claridad en el desarrollo y permite realizar modificaciones en una capa sin afectar directamente a las demás.

Sin embargo, dado que se trata de un sistema distribuido con múltiples bibliotecas operando en distintos puntos del país y compartiendo datos en tiempo real el patrón cliente-servidor también resulta esencial, este permite establecer una instancia central (servidor) que administra los servicios y comunicaciones, mientras que los dispositivos de los usuarios y administradores (clientes) interactúan con el sistema para consultar disponibilidad de libros, hacer reservas, gestionar préstamos, etc.

Además, el diseño debe contemplar futuras ampliaciones del sistema, como nuevas bibliotecas integradas o funcionalidades adicionales, por eso, la combinación de capas y cliente-servidor ofrece una solución robusta, escalable y organizada, que responde tanto al componente técnico como al enfoque de sostenibilidad operativa que requiere el proyecto.

Respuesta pregunta 2

Figura 1: Diagrama UML



Fuente: Elaboración propia (2025) basada en la pregunta 4 del proyecto 2.

Respuesta 3

Historia Usuario	COMO UN:	QUIERO:	CON EL FIN DE:	CRITERIOS DE ACEPTACIÓN
Registrar usuario/lector afiliado	Gestor de biblioteca	Registrar los datos personales de un nuevo lector afiliado	Que pueda acceder a los servicios del sistema y gestionar sus préstamos y reservas	• El sistema permite ingresar nombre, dirección, teléfono, correo electrónico y tipo de afiliación. • Los datos se almacenan correctamente en la base de datos.
Registrar préstamo	Lector afiliado	Solicitar el préstamo de un libro disponible en cualquier biblioteca	Acceder al material bibliográfico desde cualquier punto de la red	• El sistema verifica disponibilidad del libro. • Se registra fecha de préstamo y fecha estimada de devolución. • El préstamo queda vinculado al usuario.

Registrar personal biblioteca	Administrador del sistema	Agregar información del personal que atiende cada biblioteca	Asegurar el correcto acceso según roles y permisos definidos	• Se ingresan datos como nombre, cargo, biblioteca asignada y credenciales. • El personal registrado puede acceder según el rol definido por el sistema.
Registrar pago	Lector afiliado	Realizar el pago de afiliación, préstamo o multa	Completar los trámites requeridos para el uso de los servicios	• El sistema permite seleccionar tipo de pago (afiliación, préstamo, multa). • Se realiza el pago de forma segura. • Se genera un comprobante.
Generar notificaciones	Sistema (automático)	Enviar alertas de renovación, devolución o disponibilidad de libros	Mantener informado al usuario sobre el estado de sus préstamos	• El sistema envía notificaciones vía email y móvil. • Se genera una notificación al acercarse la fecha

				de devolución. • Se activa una alerta si el libro está disponible.
--	--	--	--	---

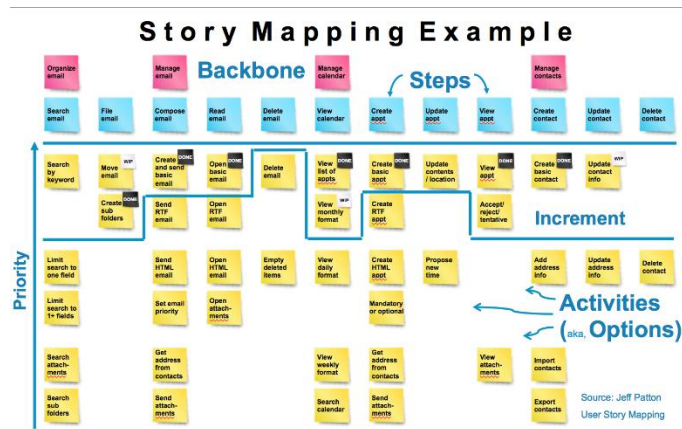
Fuente: Elaboración propia (2025) basada en el caso de estudio.

Respuesta 4

El Story Mapping es una técnica utilizada en metodologías ágiles que permite comprender y estructurar el desarrollo de un producto desde la perspectiva del usuario por lo que es importante entender que esta técnica busca responder a preguntas clave como: ¿qué funcionalidades necesita el usuario?, ¿qué valor aporta cada acción?, ¿cómo se distribuyen estas funcionalidades a lo largo del tiempo?.

Por ende "es una técnica ágil que permite organizar el Product Backlog en dos dimensiones con la intención de visualizar las funcionalidades del producto por una parte y por la otra los releases del producto."(Araneda, 2022, párr.01) por lo que se observa que esta doble dimensión es fundamental, ya que permite priorizar las funcionalidades según su relevancia, y al mismo tiempo, definir cuándo se irán liberando esas funcionalidades, lo que facilita una planificación más realista y centrada en el valor entregado. Además, el story mapping promueve una conversación activa entre desarrolladores, usuarios y stakeholders, contribuyendo a validar y ajustar las expectativas continuamente y a nivel visual, suele representarse como un mapa horizontal donde cada actividad del usuario se descompone en tareas concretas (verticalmente), organizadas por bloques funcionales que se agrupan en releases.

Ilustración 4: Story Mapping



Fuente: Tomada de *Story Telling with Story Mapping* [Imagen].

(2018). Pinterest. <https://i.pinimg.com/originals/26/bb/fa/26bbfac7d1623a0b610c5bf6204580e7.png>

Respuesta 5

Tabla 2: Manejo de riesgos

Riesgo	Probabilidad	Impacto	Acciones de Mitigación
1. Falla en la visualización en tiempo real de disponibilidad de libros	Alta	Alto	• Validar previamente la integración de datos entre bibliotecas. • Implementar mecanismos de sincronización constante. • Usar almacenamiento en caché controlado.
2. Interrupciones del servidor central de lógica de negocio	Media	Alto	• Aplicar arquitectura con tolerancia a fallos y alta disponibilidad. • Implementar balanceadores de carga. • Programar respaldos automáticos.

3. Dificultades en la compatibilidad multiplataforma de la aplicación móvil y web	Alta	Medio	• Usar frameworks responsivos y de desarrollo cruzado. • Realizar pruebas de compatibilidad en múltiples dispositivos y navegadores.
4. Vulnerabilidad en la seguridad del sistema frente a accesos no autorizados	Alta	Alto	• Implementar autenticación de dos factores. • Establecer control granular de roles y permisos. • Auditar regularmente los registros del sistema.
5. Retrasos en el procesamiento de pagos o errores en la integración de servicios de cobro en línea	Media	Medio	• Validar APIs de pago con certificados de seguridad. • Realizar pruebas end-to-end antes del despliegue. • Mantener soporte técnico de emergencia.

Fuente: Elaboración propia (2025) basada en el caso de estudio.

Respuesta 6

Los principios **SOLID** representan una guía fundamental para la creación de software limpio, escalable y sostenible a largo plazo, estos principios orientan la estructuración del código bajo criterios de coherencia, simplicidad y alta cohesión entre módulos, son especialmente útiles en ambientes colaborativos y en sistemas que evolucionan constantemente.

En ese contexto, “El acrónimo SOLID representa los cinco principios que facilitan el proceso de desarrollo lo que facilita el mantenimiento y la expansión del software.”(Muñoz, 2024, párr.05), esta afirmación permite comprender que, al aplicar estos principios, no solo se logra una mejor organización del código, sino también un producto más resistente a los cambios, con menos dependencia entre clases, y mayor facilidad para introducir nuevas funcionalidades sin comprometer la integridad del sistema.

Principio de Responsabilidad Única (Single Responsibility Principle)

Una de las causas más frecuentes de errores en proyectos grandes es la mezcla de responsabilidades dentro de una misma clase, esta práctica dificulta el mantenimiento y el control del comportamiento del sistema, de acuerdo con su definición, “El Principio de Responsabilidad Única dice que una clase debe hacer una cosa y, por lo tanto, debe tener una sola razón para cambiar.”(Rodríguez, 2022, párr.01), este enfoque permite que cada clase se mantenga autónoma, centrada en una función clara, facilitando su reutilización y su testeo, además, al aplicar este principio, el código se vuelve más predecible y menos vulnerable ante modificaciones inesperadas.

Principio de Abierto/Cerrado (Open/Closed Principle)

Antes de considerar este principio, hay que tener presente que en sistemas vivos, los cambios de requerimientos son constantes, por eso, es vital que las clases puedan adaptarse sin poner en riesgo funcionalidades ya implementadas, en este sentido, “El principio de apertura y cierre exige que las clases deban estar abiertas a la extensión y cerradas a la modificación.”(Rodriguez, 2022, párr.04), al aplicar esta idea, se promueve una codificación que permite agregar nuevos comportamientos mediante herencia, interfaces o funciones externas, sin alterar el código existente.

Principio de Sustitución de Liskov (Liskov Substitution Principle)

El desarrollo orientado a objetos se basa en la relación entre clases y subclases, pero si estas no respetan la lógica de su clase base, pueden provocar resultados inesperados, en este caso, “El Principio de Sustitución de Liskov establece que las subclases deben ser sustituibles por sus clases base.”(Rodriguez, 2022, párr.06), este principio garantiza que cualquier instancia de una subclase pueda funcionar correctamente en lugar de su clase madre, sin alterar el funcionamiento general del sistema.

Principio de Inversión de Dependencia

Este principio busca cambiar la dirección tradicional en la que fluye la dependencia dentro de un sistema, normalmente, los módulos de alto nivel dependen de los de bajo nivel para ejecutar tareas específicas; sin embargo, esta dependencia directa puede generar acoplamiento rígido y dificultar la evolución del sistema, al aplicar la inversión de dependencia, se introduce una capa de abstracción como interfaces o clases abstractas que permite que ambos niveles dependan de definiciones comunes. Esto facilita que la lógica del sistema se mantenga estable, incluso cuando se actualiza o reemplaza una funcionalidad concreta.

Principio de inversión de dependencia (D - Dependency Inversion Principle)

La formulación técnica de este principio dentro del acrónimo SOLID pone énfasis en que las dependencias deben orientarse hacia abstracciones, y no hacia implementaciones concretas, esto significa que los detalles deben depender de las generalidades, y no al revés, al aplicar este principio, se logra que el código sea más flexible, reutilizable y testeable, ya que los módulos de más alto nivel pueden operar independientemente de cómo se implementen los servicios reales, en sistemas como RedBiln, esto se vuelve especialmente útil en componentes como el servidor de notificaciones o el gestor de pagos, que pueden ser diseñados para intercambiar fácilmente el proveedor tecnológico sin comprometer la lógica del sistema.

Respuesta 7

El estándar ISO/IEC 25010:2011 representa una evolución significativa en la forma de conceptualizar y evaluar la calidad del software, surgido como parte del conjunto de normas SQuaRE, este modelo busca establecer una definición sistemática y evaluable de lo que se entiende por “producto software de calidad”, antes de profundizar en sus objetivos, es importante entender que esta norma proporciona un marco conceptual que permite al equipo de desarrollo, a los evaluadores y a los usuarios hablar en términos comunes cuando se discute la calidad del sistema.

En este sentido, “En este modelo se determinan las características de calidad que se van a tener en cuenta a la hora de evaluar las propiedades de un producto software determinado.”(Iso 25010, 2023., párr.02), lo cual implica un enfoque más preciso y verificable sobre atributos como la seguridad, la eficiencia, la mantenibilidad y la experiencia del usuario, al utilizar esta estructura, los equipos de desarrollo pueden establecer criterios concretos para medir si el software cumple con los objetivos técnicos y operativos planteados desde el inicio del proyecto.

Más allá de su formulación técnica, el estándar tiene un alcance transversal. “Este modelo se aplica en diversas áreas de la ingeniería de software, principalmente para especificar los requisitos de calidad, medir el rendimiento de los productos y evaluar su idoneidad en entornos específico”(Glarity, 2025, párr.04), esto refuerza su utilidad en casos como RedBiln, donde se requiere garantizar la calidad en distintos frentes: desde el uso en dispositivos móviles, hasta la seguridad en el manejo de datos personales y la consistencia del sistema entre bibliotecas.

Conclusión

En conclusión, el estudio comparativo de los patrones arquitectónicos permitió identificar cómo la elección estructural influye directamente en el rendimiento, escalabilidad y mantenimiento de sistemas como RedBiln, al analizar modelos como MVC, arquitectura en capas y cliente-servidor, se logró comprender que la combinación de estos patrones puede responder de manera óptima a las necesidades distribuidas del sistema, que opera en más de treinta bibliotecas.

Cabe destacar que la elaboración del diagrama de despliegue UML representó un ejercicio significativo de abstracción y visualización técnica, al integrar capas físicas, servidores especializados y dispositivos cliente, se logró clarificar cómo interactúan los subsistemas lógicos con los elementos reales del entorno informático, este tipo de modelado es esencial en proyectos que involucran procesamiento distribuido y comunicación entre aplicaciones móviles y web.

Por ende puede afirmar que la redacción de historias de usuario bajo enfoque ágil favoreció la planificación funcional del sistema desde una perspectiva empática y orientada al valor, cada historia fue elaborada considerando las necesidades reales de actores como lectores afiliados, gestores bibliotecarios y administradores del sistema, al definir objetivos claros y criterios de aceptación verificables, se promovió un diseño centrado en la usabilidad, la simplicidad de la interfaz y la eficiencia operativa.

Del análisis de riesgos emergió una visión crítica sobre las vulnerabilidades que podrían comprometer el éxito del sistema, identificar escenarios como fallas en la visualización en tiempo real, caídas del servidor o incompatibilidades multiplataforma, permitió al estudiante establecer medidas de mitigación proactivas y realistas,, estas estrategias incluyen el uso de almacenamiento en caché, pruebas de

compatibilidad cruzada, sistemas de seguridad con autenticación doble y respaldo automático de datos.

Finalmente, al explorar el estándar ISO/IEC 25010:2011 se logró vincular teoría y práctica en el desarrollo de sistemas de calidad, este modelo permitió delimitar atributos evaluables como usabilidad, seguridad, eficiencia operativa y satisfacción en el uso. Al aplicarlos al contexto de RedBiln, se comprendió que no basta con construir un sistema funcional, sino que debe responder a criterios de confiabilidad y aceptación a largo plazo.

Bibliografía

- Araneda, O. (2022). *User Story Mapping ¿Qué es y para qué sirve?* Atenos. <https://atenos.com/agile/user-story-mapping-que-es-y-para-que-sirve/>
- García, M. (2017, 5 de octubre). *MVC (Modelo-Vista-Controlador): ¿qué es y para qué sirve?* Blog de tecnologías de la información. <https://codingornot.com/mvc-modelo-vista-controlador-que-es-y-para-que-sirve>
- García de Zúñiga, F. (2025). *Arquitectura cliente servidor: Qué es, tipos y ejemplos | Blog de Arsys | Blog de Arsys*. Arsys. <https://www.arsys.es/blog/todo-sobre-la-arquitectura-cliente-servidor>
- Glarity. (2025). *¿Qué es la norma ISO/IEC 25010:2011 y cómo se aplica en la evaluación de calidad de sistemas y software?* <https://askai.glarity.app/es/search/¿Qué-es-la-norma-ISO-IEC-25010-2011-y-cómo-se-aplica-en-la-evaluación-de-calidad-de-sistemas-y-software>
- Iso 25010. (2023). PORTAL ISO 25000. <https://www.iso25000.com/index.php/normas-iso-25000/iso-25010>
- Muñoz, S. (2024). *SOLID: Qué es y cuáles son los 5 principios de la programación orientada a objetos (POO) | alura cursos online*. Alura. <https://www.aluracursos.com/blog/solid>
- Pantaleo, G., & Rinaudo, L. (2015). *Ingeniería de software*. Alfaomega Grupo Editor Argentino.
- RJ Code Advance. (2022). *Arquitectura en capas – análisis completo + tradicional vs modernas, DDD, DIP (cap 5) – RJ code advance*. RJ Code Advance Fast and

easy learning. <https://rjcodeadvance.com/patrones-de-software-arquitectura-en-capas-analisis-completo-ejemplo-ddd-parte-5/>

Rodriguez, D. S. (2022, 28 de noviembre). *Los principios SOLID de programación orientada a objetos explicados en Español sencillo*. freeCodeCamp.org. <https://www.freecodecamp.org/espanol/news/los-principios-solid-explicados-en-espanol/>